Pong

Curses

In Terminal Window Creator

```python
def main(screen):
    screen.keypad(True)
    screen.nodelay(True)

    curses.curs_set(0)  # hide

    y, x = 0, 0
    h, w = 30, 100

    win = curses.newwin(h, w, y, x)
    win.border()

curses.wrapper(main)
```

Basic Terminal Draw Capability

```python
def DrawPoints(win: curses.window):
    win.addstr(2, 35, f'Player 1) {points.x}   Player 2) {points.y}')
```

```python
def DrawPaddles(win: curses.window):
    # Paddle 1
    for i in range(paddle1.height):
        win.addstr(currentPaddlePos.x + i, paddle1Posi.x, paddle1.sprite)
    # Paddle 2
    for i in range(paddle2.height):
        win.addstr(currentPaddlePos.y + i, paddle2Posi.x, paddle2.sprite)
```

Motion

Fixed Update Loop

```
while(running):
    win = SetUp()

    running = CheckForInputs(screen)

    FixedUpdate(win)

    RefreshScreen(win)
```

Redrawing Moving Objects at New Coordinates

```
def CalculateBallPos():
    velocity = direction * ball.speed * timeStep

    ball.realX += velocity.x
    ball.realY += velocity.y

    currentBallPos.x = ball.roundedX()
    currentBallPos.y = ball.roundedY()
```

```
def FixedUpdate(win: curses.window):
    CalculatePaddlePositions()
    DrawPaddles(win)

    CalculateBallPos()
    CheckWallCollision()
    DrawBall(win)

    DrawPoints(win)

    time.sleep(timeStep)
```

```
def DrawBall(win: curses.window):
    win.addstr(currentBallPos.y, currentBallPos.x, ball.sprite)
```

Inputs

Input Check Loop

```python
def CheckForInputs(screen):
    global running

    keys = []

    while True:
        key = screen.getch()
        if key == -1:
            break
        keys.append(key)

    if 10 in keys or 13 in keys:
        running = False
        return running

    # Paddle 1
    if ord(up_1) in keys:
        paddleVertDirection.x = -1

    elif ord(down_1) in keys:
        paddleVertDirection.x = 1

    else:paddleVertDirection.x = 0

    # Paddle 2
    if up_2 in keys:
        paddleVertDirection.y = -1

    elif down_2 in keys:
        paddleVertDirection.y = 1

    else:paddleVertDirection.y = 0

    CheckPaddleWallCollision()

    return running
```

Custom Player Keybinds

```python
#Player 1 Keybinds
up_1, down_1 = 'w', 's'
#Player 2 Keybinds
up_2, down_2 = curses.KEY_UP, curses.KEY_DOWN
```

Collision

Collision Checking

```
def CheckWallCollision():
    # Collision check with paddle 1
    if currentBallPos.x == paddle1Posi.x:
        if currentBallPos.y == currentPaddlePos.x or currentBallPos.y - currentPaddlePos.x <= paddle1.height and currentBallPos.y - current
            direction.x *= -1


    # Collision check with paddle 2
    if currentBallPos.x + 1 == paddle2Posi.x:
        if currentBallPos.y == currentPaddlePos.y or currentBallPos.y - currentPaddlePos.y <= paddle2.height and currentBallPos.y - current
            direction.x *= -1


    if currentBallPos.x + len(ball.sprite) - 1 >= w - 1:
        ResetBall()
        points.x += 1
    elif currentBallPos.x <= 0:
        ResetBall()
        points.y += 1

    elif currentBallPos.y >= h - 1 or currentBallPos.y <= 0:
        # hitting/exiting bottom or top of screen
        direction.y *= -1

        # recalculate position with new direction
        CalculateBallPos()
```

Utils

Ball

```python
class Ball:
    def __init__(self, realX: float, realY: float, speed: float, sprite = 'O'):
        self.realX = realX
        self.realY = realY
        self.speed = speed
        self.sprite = sprite

    def roundedX(self):
        return round(self.realX)

    def roundedY(self):
        return round(self.realY)
```

Vector2

```python
class Vector2:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def zero(self):
        self.x = 0
        self.y = 0

    def one(self):
        self.x = 1
        self.y = 1

    def __mul__(self, other):
        return Vector2(self.x * other, self.y * other)

    def __add__(self, other):
        return Vector2(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"{self.x}, {self.y}"
```

Paddles

```python
class Paddle:
    def __init__(self, realY: float, height: int, sprite = '|'):
        self.realY = realY
        self.height = height
        self.sprite = sprite

    def roundedY(self):
        return round(self.realY)
```